Page 1: Introduction to Java

Definition

Java is a high-level, class-based, strongly typed programming language. Source code is compiled into bytecode, which runs on a Java Virtual Machine (JVM).

Features

Platform independence, object-oriented design, automatic memory management, exception handling, multithreading, portability, and a rich standard library.

Uses

Enterprise applications, backend services, banking systems, distributed systems, desktop utilities, and cloud-based applications.

First program

```
public class Main {
public static void main(String[] args) {
System.out.println("Hello, Java!");
}
}
```

Execution

Save as Main.java, compile using javac Main.java, then run using java Main. The compiler produces Main.class bytecode.

Page 2: JDK, JRE, JVM and Execution

JDK

Java Development Kit. Includes development tools such as javac, the launcher, debugging utilities, and runtime components.

JRE

Java Runtime Environment. Provides the libraries and components required to run Java applications. Modern distributions commonly use the JDK distribution for both development and runtime.

JVM

Java Virtual Machine loads classes, verifies bytecode, manages runtime memory, and executes bytecode through interpretation and Just-In-Time compilation.

Execution flow

Source file (.java) → compiler → bytecode (.class) → class loader → bytecode verifier → JVM execution engine → operating system.

Portability

Bytecode is portable, but a compatible JVM must be available on the target operating system and architecture.

Page 3: Variables, Data Types and Literals

Variables

A variable is a named storage location. Example: int age = 24;. Local variables must be initialized before use.

Primitive types

byte, short, int, long, float, double, char and boolean. Integer types store whole numbers; float and double store floating-point values.

Reference types

Classes, arrays, interfaces, enums and records are reference types. A reference may point to an object or hold null.

Examples

int count = 10; long population = 8000000000L; double rate = 2.5; char grade = "A".charAt(0); boolean active = true; String name = "Java";

Constants and casting

Use final to prevent reassignment. Widening conversions are often automatic; narrowing conversions may lose data and require a cast, for example (int) 3.9 produces 3.

Page 4: Operators and Expressions

Arithmetic

+, -, *, /, %. Integer division truncates the fractional part: 7 / 2 is 3, while 7.0 / 2 is 3.5.

Comparison

==, !=, <, >, <= and >= compare values. For objects, == compares references; equals() usually compares logical content when implemented appropriately.

Logical operators

&& means AND, || means OR, and ! means NOT. The && and || operators short-circuit when the result is already known.

Assignment and unary

=, +=, -=, *=, /=, ++ and --. Prefix increment changes the value before yielding it; postfix yields the old value first.

Conditional operator

int max = (a > b) ? a : b;. Use parentheses to make complex expressions easier to read.

Page 5: Control Flow Statements

if / else

Use if, else if and else to select a branch based on a boolean condition.

switch

switch branches on supported types such as int-compatible values, String and enum values. Modern Java versions also support switch expressions.

Loops

for is useful for counted iteration; while checks the condition before each iteration; do-while executes at least once.

Loop control

break exits a loop or switch. continue skips to the next loop iteration. Use labeled forms sparingly.

Example

for (int i = 1; i <= 5; i++) { System.out.println(i); }

Good practice

Make sure loop conditions eventually become false. Use braces for branches and loops to avoid accidental logic errors.

Page 6: Arrays and Varargs

Arrays

An array stores a fixed number of values of one declared component type. Indexes start at 0; the last valid index is length - 1.

Declaration

int[] numbers = {10, 20, 30}; or int[] values = new int[5];. Default elements are 0 for numeric types, false for boolean, and null for references.

Utilities

Use numbers.length for size and java.util.Arrays for sort(), binarySearch(), equals(), fill() and toString(). Binary search requires sorted data.

Multidimensional arrays

int[][] matrix = {{1, 2}, {3, 4}}; is an array of arrays. Java also allows jagged arrays whose rows have different lengths.

Varargs

A parameter such as sum(int... values) accepts zero or more int arguments. Inside the method, values behaves as an array; varargs must be the final parameter.

Page 7: Strings and StringBuilder

String immutability

String represents text and is immutable. Methods that appear to modify text return a new String. String literals may be reused from the string pool.

Comparison

Use "java".equals(input) to compare contents safely. equalsIgnoreCase() ignores case. Avoid == for string content comparison.

Common methods

length(), charAt(), substring(), contains(), indexOf(), startsWith(), endsWith(), replace(), split(), strip(), trim(), and toUpperCase().

StringBuilder

Use StringBuilder when repeatedly assembling text, especially in loops. It is mutable and usually more efficient than repeated concatenation.

Example

StringBuilder sb = new StringBuilder(); sb.append("Core ").append("Java"); System.out.println(sb.toString());

Page 8: Methods and Parameter Passing

Method structure

A method has a return type, name, parameters and body. Use void if it returns no value; return supplies a result from a non-void method.

Pass-by-value

Java always passes arguments by value. For objects, the copied value is a reference, so a method can mutate the object but cannot reassign the caller's variable.

Overloading

Methods may share a name if their parameter lists differ by number, types or order. Return type alone cannot distinguish overloaded methods.

Recursion

A recursive method calls itself. It needs a base case and progress toward that case; excessive recursion can cause StackOverflowError.

Design

Keep methods focused, use meaningful names, validate inputs where necessary, and avoid very long methods.

Page 9: Object-Oriented Programming

Class and object

A class defines state and behavior. An object is an instance of a class; fields represent state and methods represent behavior.

Encapsulation

Keep fields private and provide controlled methods to protect object invariants and reduce coupling.

Inheritance

A subclass extends a superclass and can reuse or specialize behavior. Java classes support single inheritance, while a class can implement multiple interfaces.

Polymorphism

A superclass or interface reference can refer to objects of different implementations. Overridden instance methods are selected at runtime.

Abstraction

Expose essential operations and hide implementation details, commonly using interfaces and abstract classes.

Page 10: Constructors, this and static

Constructors

A constructor initializes an object. It has the class name and no return type. If none is declared, the compiler may provide a default no-argument constructor.

Constructor chaining

Use this(...) to call another constructor in the same class or super(...) to invoke a superclass constructor. The invocation must be first under traditional syntax.

this

this refers to the current object. Example: this.name = name distinguishes the field from a parameter.

static

A static field belongs to the class and is shared. A static method can be called without an instance but cannot directly access instance members without an object.

Initialization

Field initializers and instance initializers participate in object creation. Keep constructors straightforward and avoid complicated side effects.

Page 11: Inheritance, Overriding and super

extends

A subclass inherits accessible members from a superclass. Constructors are not inherited, and Java does not allow a class to extend multiple classes.

Overriding

A subclass provides a compatible implementation of an inherited instance method. Use @Override to let the compiler detect signature mistakes.

super

super accesses superclass members or invokes a superclass constructor. It is useful when extending existing behavior.

Rules

An overriding method cannot reduce visibility. Static methods are hidden rather than overridden. final methods cannot be overridden and final classes cannot be subclassed.

Composition

Prefer composition when one object uses or contains another object instead of having a genuine “is-a” relationship.

Page 12: Abstraction, Interfaces and Polymorphism

Abstract class

An abstract class cannot be instantiated directly. It may contain abstract methods, concrete methods, fields and constructors.

Interface

An interface defines a contract. Depending on Java version, interfaces can include abstract, default, static and private methods.

Runtime polymorphism

Shape shape = new Circle(); lets code use the Shape contract while Circle provides the overridden implementation.

Overloading vs overriding

Overloading is resolved at compile time from parameter lists. Overriding uses runtime dispatch for instance methods.

Design guidance

Use interfaces to create meaningful contracts and abstraction to separate what a component does from how it does it.

Page 13: Packages, Access Modifiers and Enums

Packages

Packages organize related types and reduce naming conflicts. A package declaration normally appears at the beginning of the source file.

Imports

import allows types to be used by simple names; it does not copy source code. java.lang types are automatically available.

Access levels

public: broadly accessible; protected: same package and subclasses subject to access rules; package-private: same package; private: within the declaring class.

Enums

An enum defines a fixed set of constants, such as enum Status { NEW, RUNNING, DONE }. Enums may also have fields and methods.

Naming conventions

Class names use PascalCase, methods and variables use camelCase, and constants commonly use UPPER_SNAKE_CASE.

Page 14: Exception Handling

Hierarchy

Throwable is the root of Java's throwable hierarchy. Error generally indicates serious system-level problems; Exception represents conditions applications may handle.

Checked and unchecked

Checked exceptions must be caught or declared. RuntimeException and its subclasses are unchecked and often indicate invalid input or programming errors.

try/catch/finally

Put risky operations in try, handle specific exceptions in catch, and use finally for cleanup that must be attempted.

throw and throws

throw raises a throwable object. throws declares possible checked exceptions in a method signature.

Try-with-resources

Use try (Resource r = ...) with AutoCloseable resources such as streams; they close automatically even when an exception occurs.

Best practice

Do not swallow exceptions. Preserve causes, provide useful context, and avoid using exceptions for ordinary control flow.

Page 15: Collections Framework

Interfaces

List preserves sequence, Set enforces uniqueness, Queue supports processing elements, and Map stores key-value pairs. Map is not a subtype of Collection.

List

ArrayList is a common default and offers fast indexed access. LinkedList supports deque operations but often has more memory overhead.

Set

HashSet has no guaranteed iteration order; LinkedHashSet preserves insertion order; TreeSet maintains sorted order.

Map

HashMap is widely used and has no guaranteed iteration order. LinkedHashMap preserves encounter order; TreeMap sorts keys.

Iteration

Use enhanced for loops or iterators. Follow the iterator's remove() contract when removing during iteration.

Choosing a collection

Consider ordering, uniqueness, lookup speed, insertion/removal patterns and concurrency requirements.

Page 16: Generics, equals/hashCode and Ordering

Generics

Generics provide compile-time type safety, for example List<String>. They reduce casts and catch many type errors before runtime.

Type parameters

A generic class may declare <T>. Bounds such as <T extends Number> restrict types. Wildcards include ?, ? extends T and ? super T.

equals and hashCode

If two objects are equal according to equals(), they must return the same hashCode(). This contract is essential for HashMap and HashSet.

Comparable and Comparator

Comparable<T> defines natural order through compareTo(). Comparator<T> defines an external or alternative order through compare().

Type erasure

Generic type information is mostly erased at runtime. You generally cannot create new T[] directly or use instanceof List<String>.

Page 17: File Handling and I/O

Packages

java.io provides stream and reader/writer APIs. java.nio.file provides Path, Files and modern file-system operations.

Bytes vs characters

InputStream and OutputStream handle bytes. Reader and Writer handle characters and are suitable for text with an explicit charset.

Path and Files

Use Path to represent paths and Files methods to read, write, copy, move, list and check paths. UTF-8 is a sensible explicit choice for text.

Buffered I/O

BufferedReader and BufferedWriter improve efficiency for many text operations. Use try-with-resources to close resources reliably.

Serialization

Native Java serialization has security and compatibility concerns. For many data-exchange scenarios, JSON or another explicit format is preferable.

Page 18: Multithreading and Concurrency

Threads and tasks

A thread is an execution path. Define tasks with Runnable or Callable and use Thread or, commonly, ExecutorService to execute them.

Thread states

Common states include NEW, RUNNABLE, BLOCKED, WAITING, TIMED_WAITING and TERMINATED. Scheduling is controlled by the JVM and operating system.

Synchronization

synchronized protects critical sections and establishes visibility guarantees. Inconsistent lock ordering can cause deadlocks.

volatile and atomic

volatile provides visibility but does not make compound operations like count++ atomic. AtomicInteger and related classes support atomic updates.

Executors

ExecutorService manages task execution and thread reuse. Shut it down when finished; use Future to retrieve results and observe failures.

Good practice

Prefer high-level concurrency utilities, minimize shared mutable state and never assume threads execute in a particular order.

Page 19: Modern Java and JVM Memory

Lambda expressions

Lambdas concisely implement functional interfaces, for example x -> x * 2. A functional interface has one abstract method.

Streams

Streams support operations such as filter, map, sorted and collect. Intermediate operations are generally lazy; terminal operations trigger processing.

Optional

Optional represents a possibly absent return value. It should not automatically replace every nullable field or parameter.

Modern features

Records simplify data-carrier classes; switch expressions can return values. Feature availability depends on the Java version used.

JVM memory

Stack frames hold method-call data and local variables; the heap stores objects. Garbage collection reclaims unreachable objects, but not necessarily immediately.

JIT and class loading

The JVM loads classes as needed and may compile frequently executed bytecode into native machine code. Runtime performance depends on workload and warm-up.

Page 20: JDBC, Best Practices and Interview Revision

JDBC workflow

JDBC connects Java to relational databases. Obtain a Connection, prepare a statement, bind parameters, execute, process results and close resources.

PreparedStatement

Use placeholders (?) and bind values. Parameterized statements help prevent SQL injection for values and support cleaner query handling.

Transactions

A transaction groups operations. Disable auto-commit when needed, commit on success and roll back on failure. Keep transactions appropriately short.

Clean code

Use meaningful names, small focused methods, encapsulated fields, clear exception handling, immutable data where practical and tests for important behavior.

Interview questions

Review JVM/JRE/JDK, platform independence, == vs equals(), overloading vs overriding, interfaces vs abstract classes, exceptions, collections, HashMap, String immutability, synchronization and garbage collection.

Practice checklist

Write small programs using loops, arrays, strings, methods, classes, inheritance, exceptions and collections. For each concept, explain both the definition and a practical example.